

Embedded Systems and IoT Lab (BITE403P)

Digital Assessment – 3 8051 PROGRAMMING

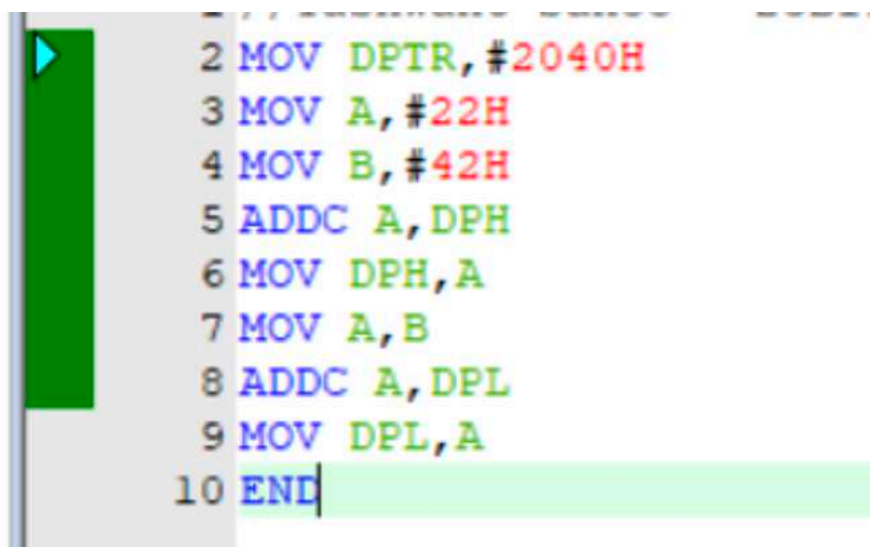
Name: Sameer Gugarwal
Reg No: 23BIT0313

8051 – ASSEMBLY LANGUAGE

1) Write an ALP for adding 2 16 bit numbers.

Code:

```
MOV DPTR,#2040H
MOV A,#22H
MOV B,#42H
ADDC A,DPH
MOV DPH,A
MOV A,B
ADDC A,DPL
MOV DPL,A
END
```



Registers	
Register	Value
[-] Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
[-] Sys	
a	0x82
b	0x42
sp	0x07
sp_max	0x07
dptr	0x4282
PC \$	C:0x0012
states	10
sec	0.00000500
[+] psw	0x04

2) Write an ALP for adding 2 8 bit numbers

Code:

MOV A,#10H

MOV B,#10H

ADD A,B

END

```

2 MOV A, #10H
3 MOV B, #10H
4 ADD A, B
5 END
6

```

Registers	
Register	Value
[-] Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
[-] Sys	
a	0x20
b	0x10
sp	0x07
sp_max	0x07
dptr	0x0000
PC \$	C:0x0007
states	4
sec	0.00000200
[+] psw	0x01

3) Write an ALP for performing 16 BIT BCD addition

Code:

```

MOV DPTR, #2040H
MOV A, #22H
MOV B, #42H
ADDC A, DPL
DAA
MOV DPL, A
MOV A, B
ADDC A, DPH

```

DAA
MOV DPH,A
END

```
2 MOV DPTR,#2040H
3 MOV A,#22H
4 MOV B,#42H
5 ADDC A,DPL
6 DA A
7 MOV DPL,A
8 MOV A,B
9 ADDC A,DPH
10 DA A
11 MOV DPH,A
12 END
```

Registers	
Register	Value
[-] Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
[-] Sys	
a	0x62
b	0x42
sp	0x07
sp_max	0x07
dptr	0x6262
PC \$	C:0x0014
states	12
sec	0.00000600
[+] psw	0x01

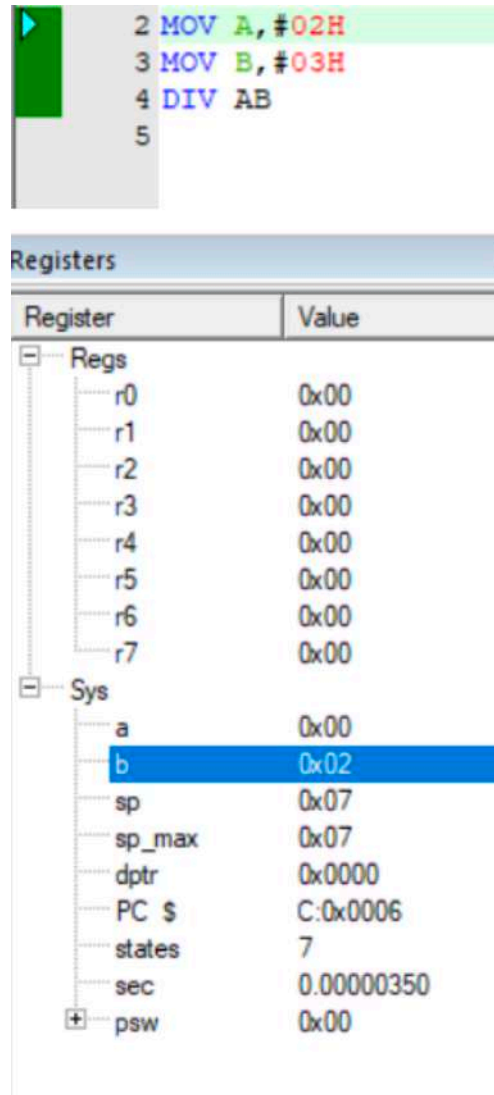
4) Write an ALP for performing division

Code:

MOV A,#02H

MOV B,#03H

DIV AB



5) Write an ALP for performing factorial

Code:

MOV DPTR,#1000H

MOVX A,@DPTR

```

MOV R1,A
MOV A,#01H
LOOP: MOV B,R1
      MUL AB
      DJNZ R1,LOOP
      MOVX @DPTR,A
      END

```

```

2 MOV DPTR,#1000H
3 MOVX A,@DPTR
4 MOV R1,A
5 MOV A,#01H
6 LOOP: MOV B,R1
7 MUL AB
8 DJNZ R1,LOOP
9 MOVX @DPTR,A
10 END
11

```

Register	Value
Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x00
b	0x00
sp	0x07
sp_max	0x07
dptr	0x1000
PC \$	C:0x000D
states	2056
sec	0.00102800
psw	0x00

Address	Value
X:0x001000:	78 00
X:0x00101B:	00 00
X:0x001036:	00 00
X:0x001051:	00 00
X:0x00106C:	00 00
X:0x001087:	00 00
X:0x0010A2:	00 00
X:0x0010BD:	00 00

Simulation

t1: 0.00002400 sec

L:10 C:7

CAP NUM: SCRL OVR: R/W

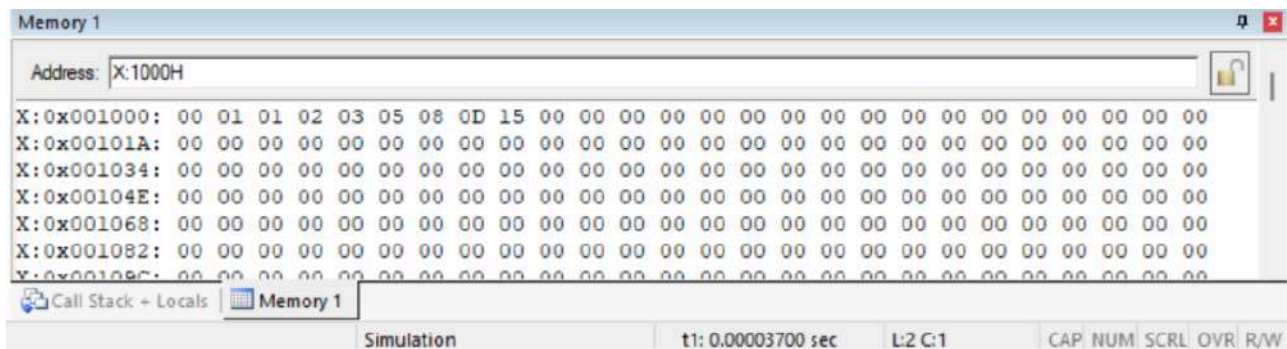
6) Write an ALP for finding the fibonacci series

Code:

```
MOV DPTR,#1000H
MOV R0,#09H
MOV A,#00
MOV B,#01H
DEC R0
DEC R0
MOVX @DPTR,A
INC DPTR
MOV A,B
MOVX @DPTR,A
MOV A,#00H
MOV B,#01H
LOOP: ADD A,B
INC DPTR
MOVX @DPTR,A
XCH A,B
DJNZ R0,LOOP
END
```

```
2 MOV DPTR,#1000H
3 MOV R0,#09H
4 MOV A,#00H
5 MOV B,#01H
6 DEC R0
7 DEC R0
8 MOVX @DPTR,A
9 INC DPTR
10 MOV A,B
11 MOVX @DPTR,A
12 MOV A,#00H
13 MOV B,#01H
14 LOOP: ADD A,B
15 INC DPTR
16 MOVX @DPTR,A
17 XCH A,B
18 DJNZ R0,LOOP
19 END
20
```

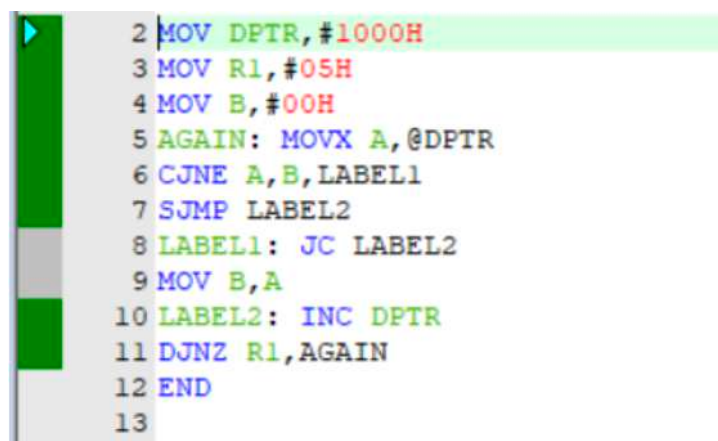
Registers	
Register	Value
Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x0d
b	0x15
sp	0x07
sp_max	0x07
dptr	0x1008
PC \$	C:0x001E
states	74
sec	0.00003700
psw	0x41



7) Write an ALP for finding the maximum number

Code:

```
MOV DPTR,#1000H
MOV R1,#05H
MOV B,#00H
AGAIN: MOVX A,@DPTR
CJNE A,B,LABEL1
SJMP LABEL2
LABEL1: JC LABEL2
MOV B,A
LABEL2: INC DPTR
DJNZ R1,AGAIN
END
```



Register	Value
Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0xfe
b	0xff
sp	0x07
sp_max	0x07
dptr	0x1005
PC \$	C:0x0015
states	57
sec	0.00002850
psw	0x81

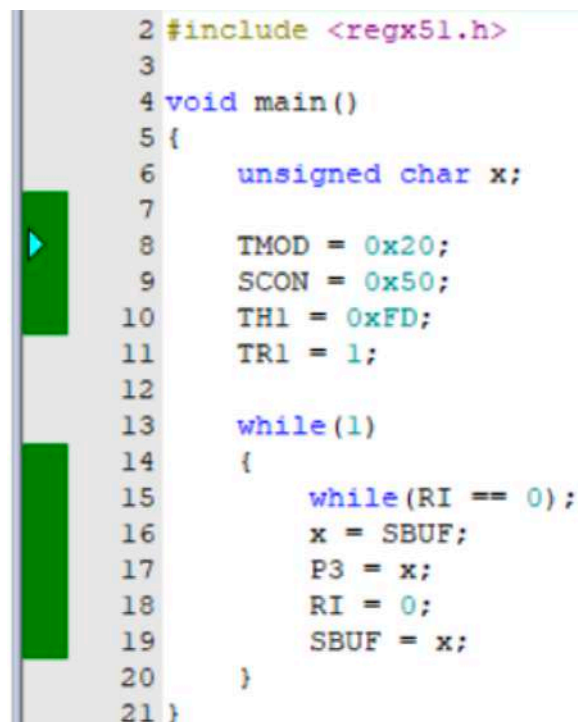
[illegible]

EMBEDDED-C-PROGRAMS

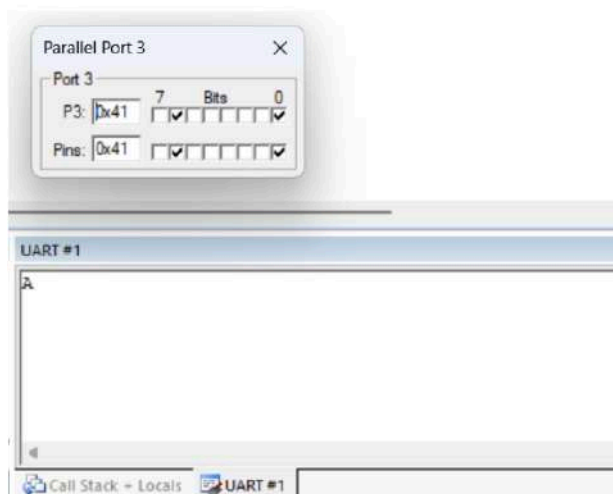
1) Receiving an ASCII Character

Code:

```
#include <regx51.h>
void main()
{
    unsigned char x;
    TMOD = 0x20;
    SCON = 0x50;
    TH1 = 0xFD;
    TR1 = 1;
    while(1)
    {
        while(RI == 0);
        x = SBUF;
        P3 = x;
        RI = 0;
        SBUF = x;
    }
}
```



```
2 #include <regx51.h>
3
4 void main()
5 {
6     unsigned char x;
7
8     TMOD = 0x20;
9     SCON = 0x50;
10    TH1 = 0xFD;
11    TR1 = 1;
12
13    while(1)
14    {
15        while(RI == 0);
16        x = SBUF;
17        P3 = x;
18        RI = 0;
19        SBUF = x;
20    }
21 }
```



2) Delay Program

Code:

```
#include <regx51.h>
```

```
void DELAY_LOOP_WAIT(unsigned int DELAY)
```

```
{
```

```
    unsigned int x,y;
```

```
    for(x=0;x<DELAY;x++)
```

```
    {
```

```
        for(y=0;y<120;y++);
```

```
    }
```

```
void main()
```

```
{
```

```
{
```

```
while(1)
```

```
{
```

```
    P1 = 0xFF;
```

```
    DELAY_LOOP_WAIT(500);
```

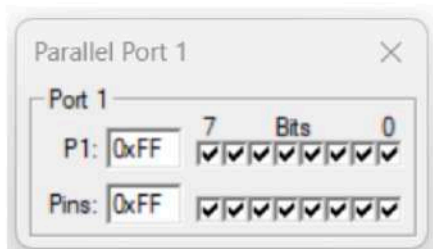
```
    P1 = 0x00;
```

```
    DELAY_LOOP_WAIT(500);
```

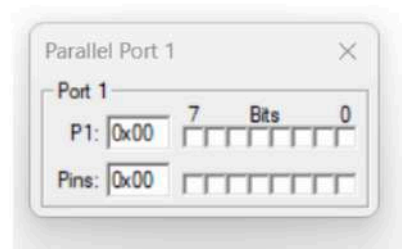
```
}
```

```
}
```

```
2 #include <regx51.h>
3
4 void DELAY_LOOP_WAIT(unsigned int DELAY)
5 {
6     unsigned int x,y;
7
8     for (x=0;x<DELAY;x++)
9     {
10         for(y=0;y<120;y++);
11     }
12 }
13
14 void main()
15 {
16     while(1)
17     {
18         P1 = 0xFF;
19         DELAY_LOOP_WAIT(500);
20
21         P1 = 0x00;
22         DELAY_LOOP_WAIT(500);
23     }
24 }
```



ON state



OFF state

3) Displaying Character from Keyboard

Code:

```
#include <reg51.h>
#include <stdio.h>
void main(void)
{
    SCON = 0x50; // UART mode 1, receiver enabled
    TMOD |= 0x20; // Timer1 mode 2 (auto reload)
    TH1 = 0xF4; // Baud rate 2400
    TR1 = 1; // Start timer1
    TI = 1; // Ready to transmit
    while(1)
    {
        unsigned char aaa;
        aaa = _getkey(); // receive character
        putchar(aaa); // send character back
    }
}
```

```
01 //Yashwant Sahoo - 23BIT0115
02 #include <reg51.h>
03 #include <stdio.h>
04
05 void main(void)
06 {
07     SCON = 0x50; // UART mode 1, receiver enabled
08     TMOD |= 0x20; // Timer1 mode 2 (auto reload)
09     TH1 = 0xF4; // Baud rate 2400
10     TR1 = 1; // Start timer1
11     TI = 1; // Ready to transmit
12
13     while(1)
14     {
15         unsigned char aaa;
16
17         aaa = _getkey(); // receive character
18         putchar(aaa); // send character back
19     }
20 }
```

4) Basic LED Program

Code:

```
#include <regx51.h>
#include "delay.c"
void main()
{
int x=0;
while(1)
{
if(x==0)
{
P0=0x00;
x=1;
DELAY_LOOP_WAIT(1000);
}
if(x==1)
{
P0=0xFF;
x=0;
DELAY_LOOP_WAIT(1000);
}
}
}
```

```
02 #include <regx51.h>
03 #include "delay.c"
04
05 void main()
06 {
07     int x=0;
08     while(1)
09     {
10         if(x==0)
11         {
12             P0=0x00;
13             x=1;
14             DELAY_LOOP_WAIT(1000);
15         }
16         if(x==1)
17         {
18             P0=0xFF;
19             x=0;
20             DELAY_LOOP_WAIT(1000);
21         }
22     }
23 }
```

Parallel Port 0

Port 0

P0: 0xFF 7 Bits 0

Pins: 0x00

Parallel Port 0

Port 0

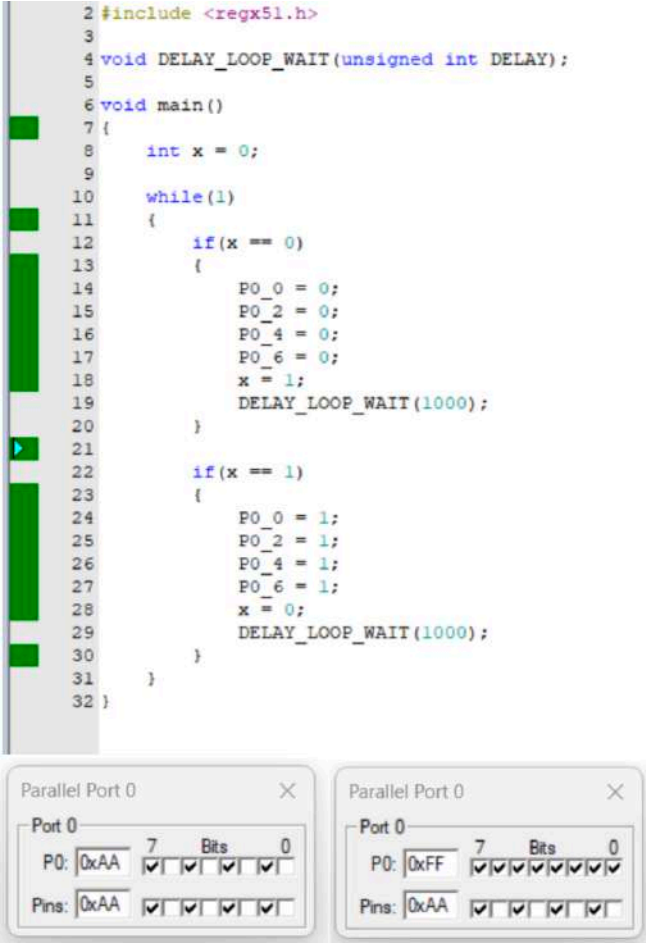
P0: 0x00 7 Bits 0

Pins: 0x00

5) Alternate LED

Code:

```
#include <regx51.h>
void DELAY_LOOP_WAIT(unsigned int DELAY);
void main()
{
int x = 0;
while(1)
{
if(x == 0)
{
P0_0 = 0;
P0_2 = 0;
P0_4 = 0;
P0_6 = 0;
x = 1;
DELAY_LOOP_WAIT(1000);
}
{
if(x == 1)
P0_0 = 1;
P0_2 = 1;
P0_4 = 1;
P0_6 = 1;
x = 0;
DELAY_LOOP_WAIT(1000);
}
}
}
```



```
2 #include <regx51.h>
3
4 void DELAY_LOOP_WAIT(unsigned int DELAY);
5
6 void main()
7 {
8     int x = 0;
9
10    while(1)
11    {
12        if(x == 0)
13        {
14            P0_0 = 0;
15            P0_2 = 0;
16            P0_4 = 0;
17            P0_6 = 0;
18            x = 1;
19            DELAY_LOOP_WAIT(1000);
20        }
21
22        if(x == 1)
23        {
24            P0_0 = 1;
25            P0_2 = 1;
26            P0_4 = 1;
27            P0_6 = 1;
28            x = 0;
29            DELAY_LOOP_WAIT(1000);
30        }
31    }
32 }
```

Parallel Port 0

Port 0

P0: 0xAA 7 Bits 0

Pins: 0xAA

Parallel Port 0

Port 0

P0: 0xFF 7 Bits 0

Pins: 0xAA

6) Basic PORT Based Arithmetic

```
2 #include <regx51.h>
3
4 void main()
5 {
6     P0 = 0x05;
7     P1 = 0x06;
8     P2 = P1 * P0;
9 }
```

Parallel Port 0		Parallel Port 1		Parallel Port 2	
Port 0		Port 1		Port 2	
P0:	0x05	P1:	0x06	P2:	0x1E
Pins:	0x05	Pins:	0x06	Pins:	0x1E

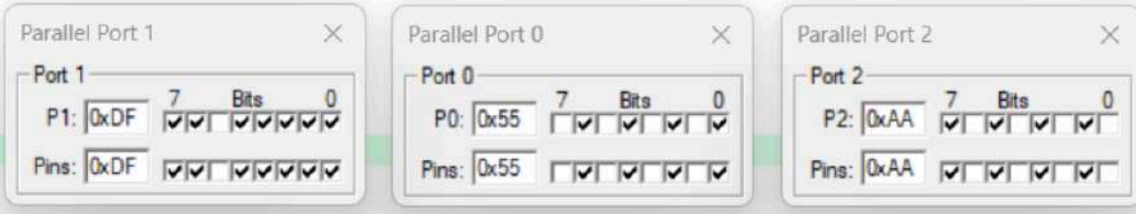
7) Port Single Bit Access

```
2 #include <regx51.h>
3
4 void main()
5 {
6     int m = P1 ^ 5;
7     P1 = m;
8 }
9
```

Parallel Port 1		Parallel Port 1	
Port 1		Port 1	
P1:	0xFF	P1:	0xFA
Pins:	0xFF	Pins:	0xFA

8) Single BIT Usage

```
2 #include <regx51.h>
3
4 sbit mybit = P1^5;
5
6 void main()
7 {
8     if(mybit == 0)
9         P0 = 0x55;
10
11     if(mybit == 1)
12         P2 = 0xAA;
13 }
```



DATA TRANSMISSION

1) Write an 8051 program to transmit character 'A' at 9600 baud using 11.0592 MHz crystal and normal baud rate.

```
2 MOV TMOD, #20H
3 MOV TH1, #0FDH
4 MOV SCON, #50H
5 SETB TR1
6
7 MOV A, #'A'
8 MOV SBUF, A
9
10 WAIT: JNB TI, WAIT
11 CLR TI
12
13 HERE: SJMP HERE
14 END
```



2) Transmit character 'B' at 19200 baud, 11.0592 MHz, where transmission happens at a rate of double the baud rate.

```
2 ORG 0000H
3
4 SMOD BIT 087H
5
6 MOV TMOD,#20H
7 MOV TH1,#0FDH
8 MOV SCON,#50H
9 SETB TR1
10
11 SETB SMOD
12
13 MOV A,#'B'
14 MOV SBUF,A
15
16 WAIT: JNB TI, WAIT
17 CLR TI
18
19 HERE: SJMP HERE
20 END
```

UART #1

B

3) Transmit 'C' at 9600 baud using 14 MHz crystal.

```
2 ORG 0000H
3
4 MOV TMOD,#20H
5 MOV TH1,#0FCH
6 MOV SCON,#50H
7 SETB TR1
8
9 MOV A,#'C'
10 MOV SBUF,A
11
12 WAIT: JNB TI, WAIT
13 CLR TI
14
15 HERE: SJMP HERE
16
17 END
```

UART #1

C

4) Transmit 9-bit data (address frame) at 9600 baud, 11.0592 MHz.

```
2 ORG 0000H
3 MOV TMOD,#20H
4 MOV TH1,#0FDH
5 MOV SCON,#0D0H
6 SETB TR1
7 MOV A,#55H
8 SETB TB8
9 MOV SBUF,A
10 WAIT: JNB TI,WAIT
11 CLR TI
12 HERE: SJMP HERE
13
14 END
```

UART #1

U